

Production of the UMLOOModeler Documentation

This document describes how the technical documentation of UMLOOModeler was produced with the support of the Codex app for Windows, using the GPT-5.5 model.

The following materials (evidence) were used to produce the documentation:

1. **Article:** the article was provided in Markdown format, together with its figures;
2. **Repository examples:** some examples available in the public repository were provided, together with the generated technical documentation example and the outputs produced by UMLOOModeler (the `clinical_shap` files);
3. **Source code:** the source code of the tool was provided;
4. **Test suite:** a test suite used to validate the tool was provided, along with its respective outputs (Mermaid, PlantUML, and JSON);
5. **Records:** records of corrections, additions, and features left for the future were provided;
6. **Other materials:** manual inputs were also provided, including the purpose and context of the project, the tool's behavior matrix, human scope decisions, and clarifications regarding possible questions the LLM had about the project.

Two approaches were identified for producing the documentation using the materials listed above. They are presented below.

First Approach

In the first approach, all materials related to UMLOOModeler would be organized and sent to the LLM at once. Through a few prompts, the materials would be processed and the LLM would be asked to generate the documentation. The document would then undergo human review. This would result in a single execution of the following pipeline:

Various materials → Organized evidence → LLM → Technical documentation → Human review

Based on the author's experience with LLMs, asking an LLM to "spit out" a technical document using this pipeline could result in degraded documentation that would require extensive manual review and correction. For this reason, this approach was not used.

Second Approach

In the second approach, the evidence is provided in separate stages through constant interaction with a human. The LLM follows and processes the construction of the documentation progressively. The project and its scope are defined; questions are answered; tests are performed to validate the tool's behavior; additional evidence is produced to validate and expand the LLM's understanding of the project; and other LLMs are used to cross-validate what is being produced. In this way, the LLM becomes familiar with the entire documentation lifecycle. The activities performed were later organized into the lifecycle illustrated in the figure below.

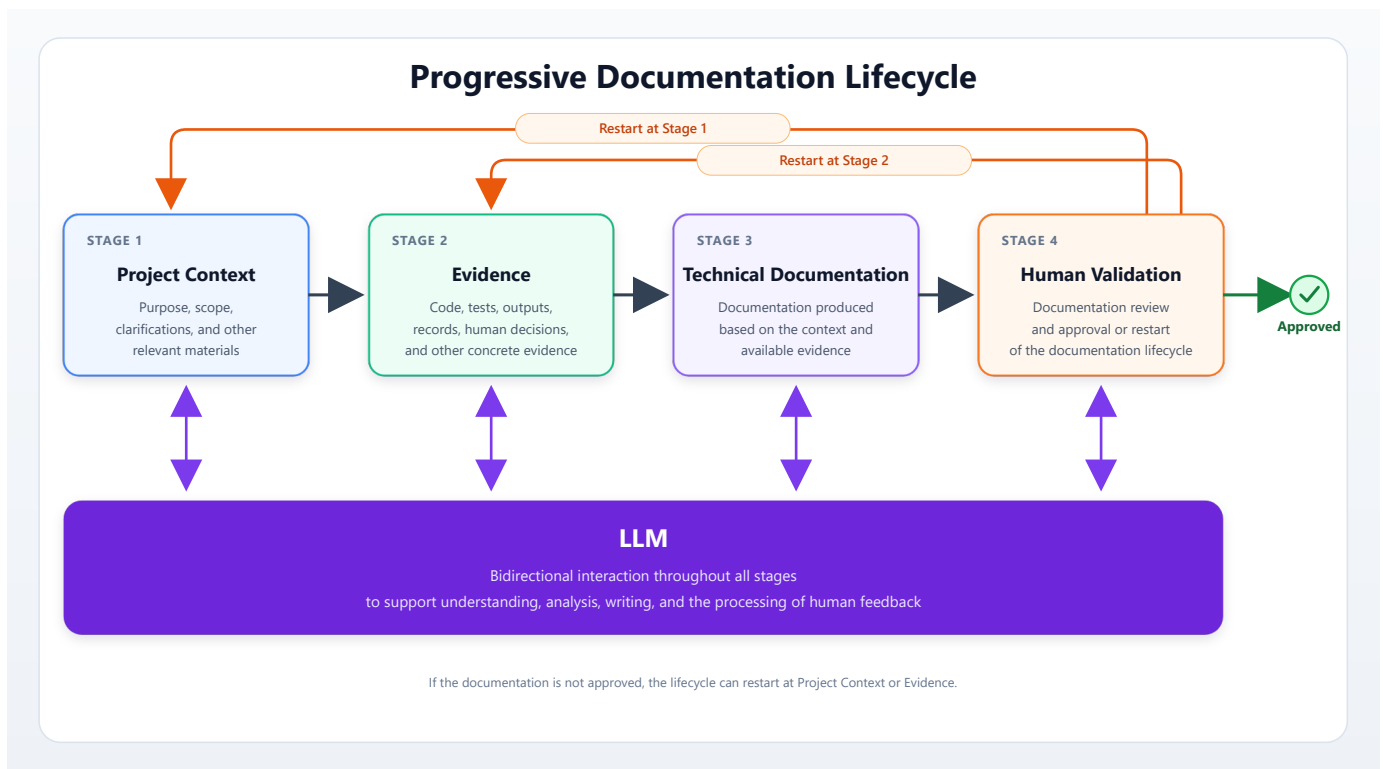


Figure 1 — Progressive documentation lifecycle. Although this figure was derived from the documentation process observed in practice, its structure resembles lifecycle models used in software development.

Lifecycle Stages

Stage 1 — Project Context. The LLM processes evidence related to the Project Context, allowing it to understand the purpose and scope of the tool.

Stage 2 — Evidence. The LLM processes evidence related to the tool itself, allowing it to understand its implementation and observable behavior.

Stage 3 — Technical Documentation. The LLM produces the technical documentation based on what was observed in the previous stages. The documentation may be customized according to the user's needs.

Stage 4 — Human Validation. A human manually reviews the documentation. If the documentation is so degraded that manual correction becomes impractical, the process can return to Stages 1 or 2 so that the context or evidence can be corrected, supplemented, or reorganized. The LLM can then be informed about the problems found and receive further guidance before generating the documentation again in Stage 3. If the generated documentation is acceptable, small corrections can be made manually and the cycle can be completed.

There is interaction between the human and the LLM in every stage. This interaction is essential for the LLM to understand the project and produce acceptable documentation.

This progressive process resulted in the technical documentation of UMLOOModeler. The resulting documentation required only a few targeted manual corrections and did not need to be rewritten entirely.

Hypotheses Raised

This experience raised two hypotheses:

1. It is believed that the LLM-assisted progressive documentation lifecycle shown in Figure 1 can be applied to projects that require software documentation. A more detailed study will be conducted to determine the kinds of projects in which it can be applied.

2. It is believed that system documentation can still be generated when evidence from Stage 1 or Stage 2 is omitted. The possible trade-off is degradation: the less relevant evidence the LLM receives, the greater the risk of unsupported statements (inferences) and degraded documentation. The more degraded the documentation is, the more time may be required for corrections and reviews.

This document reconstructs the experience and presents a preliminary representation of the process that was observed. It does not yet formalize a general methodology or demonstrate its applicability to other projects.